PROJECT IMPLEMENTATION REPORT

Agentic AI-Based Travel Planning Assistant Using LangChain

Autonomous Multi-Tool Decision Agent with Live Open-Meteo Weather Integration & Streamlit Interface

Domain:	Travel & Tourism / Agentic AI	**Date:**	November 2026
Author / Developer:	Durgesh Giri	**Framework:**	LangChain, LangGraph, Python 3.11
Live Streamlit App:	https://agentic-ai-travel-planning-assistant-6uy9jrgkprgf8afhntbunz.streamlit.app/	**Database:**	SQLite (Normalized & Indexed)
GitHub Repository:	https://github.com/durg-giri123/Agentic-AI-Travel-Planning-Assistant	**Test Suite:**	17 Passed (100% Pass Rate)

1. Executive Summary

This project delivers an enterprise-grade, agentic AI-driven travel planning assistant that autonomously synthesizes customized, realistic, and budget-optimized travel itineraries. Built on the LangChain framework, the system addresses the friction of multi-website comparison by integrating five specialized tools: Flight Search, Hotel Recommendation, Places/Attraction Discovery, Real-time Meteorological Forecasting via the free Open-Meteo API, and Itemized Budget Estimation. The solution features an autonomous multi-tool optimizer operating out-of-the-box without API key dependencies, alongside plug-and-play LLM orchestration supporting OpenAI, Google Gemini, and Groq. The architecture is backed by an indexed SQLite relational database, interactive Streamlit and CLI interfaces, and a 100% verified test suite.

2. Problem Statement & Commercial Use Cases

Problem Statement: Comprehensive trip planning necessitates evaluating interconnected variables including flight schedules, hotel pricing, neighborhood safety, attraction operating hours, real-time weather expectations, and strict budget ceilings. Traditional travelers toggle between 4 to 8 fragmented platforms, resulting in decision fatigue, unoptimized transit routes, and unrealistic day schedules.

Commercial Relevance: Major travel aggregators (MakeMyTrip, Booking.com, ClearTrip, Ixigo) are adopting agentic AI to:

- **Workload Reduction:** Automates 70%+ of front-line itinerary consultations and customer inquiries.
- **Dynamic Personalization:** Generates customized recommendations matching traveler tier (Budget, Moderate, Luxury).
- **Conversion Optimization:** Bundling flights, lodging, and local transit directly shortens booking cycles.
- **Weather-Aware Scheduling:** Prevents customer dissatisfaction by proactively re-routing outdoor activities during rainfall.

3. System Architecture & Multi-Step Agentic Workflow

The application architecture strictly separates concerns across the **Presentation Layer** (Streamlit Web App and CLI), **Cognitive Agent Layer** (LangChain ReAct / LangGraph ToolCalling & Autonomous Optimizer), **Tool Execution Layer** (5 Domain Tools), and **Persistence Layer** (SQLite Database and JSON Datasets).

Step	Phase	Operational Functionality
Step 1	Query Ingestion	Parses origin, destination, duration (3-7 days), budget tier, dates, and attraction preferences.
Step 2	Flight Search	Invokes search_flights_tool; queries database, evaluates cheapest vs fastest fares, and records justification.
Step 3	Hotel Recommendation	Invokes hotel_recommendation_tool; filters by city, star rating, nightly price, and verified amenities.

Step 4	Weather Forecast	Invokes weather_lookup_tool; calls live Open-Meteo API, fetches daily max/min temperatures and rain risk.
Step 5	Attraction Discovery	Invokes places_discovery_tool; clusters POIs by theme, scheduling outdoor spots on sunny days and museums on rainy days.
Step 6	Budget Calculation	Invokes budget_estimation_tool; computes flights + hotel nights + daily transit/food + contingency buffer.
Step 7	Synthesis & Output	Generates clean Pydantic object, formatted human-readable specification text, and Markdown exports.

4. Detailed Implementation of the 5 LangChain Tools

Each tool is implemented using the LangChain @tool decorator and explicit Pydantic argument schemas, allowing seamless introspection by ReAct agents, function-calling models, or deterministic callers.

4.1 Flight Search Tool (src/tools/flight_tool.py)

• **Input Schema:** origin, destination, sort_by ('cheapest', 'fastest', 'earliest'), max_price, preferred_airline, limit.
• **Mechanism:** Computes duration delta between departure and arrival timestamps, handles day crossovers, queries indexed SQL database records, and outputs JSON with ticket pricing and scheduled flight identifiers.
• **Coverage:** 112 direct and connecting routes connecting Delhi, Mumbai, Bangalore, Goa, Hyderabad, Chennai, Kolkata, Jaipur.

4.2 Hotel Recommendation Tool (src/tools/hotel_tool.py)

• **Input Schema:** city, min_stars (1 to 5), max_price_per_night, required_amenities, sort_by ('best_value', 'cheapest', 'highest_rated').
• **Mechanism:** Unpacks JSON-encoded amenity arrays (wifi, pool, spa, breakfast, gym), performs multi-attribute filtering, and ranks hotels using a star-to-price value coefficient.

4.3 Places Discovery Tool (src/tools/places_tool.py)

• **Input Schema:** city, categories (fort, beach, temple, museum, lake, park, market, monument), min_rating, limit.
• **Mechanism:** Queries 96 curated attractions, sorts by descending visitor rating, and groups sights into thematic clusters for morning, afternoon, and evening scheduling.

4.4 Weather Lookup Tool (src/tools/weather_tool.py & src/utils/weather_api.py)

• **Integration:** Live connection to Open-Meteo API endpoint (https://api.open-meteo.com/v1/forecast).
• **Key Attributes:** 100% Free, requires zero API keys, no monthly quota exhaustion, zero vendor lock-in.
• **WMO Code Mapping:** Translates standard WMO codes into descriptive conditions (e.g., Code 0: Clear Sky, Code 51: Light Drizzle, Code 95: Thunderstorm). Provides dynamic sightseeing advisories.
• **Resilience:** Incorporates automatic regional fallback profiles in the event of local client network disconnects.

4.5 Budget Estimation Tool (src/tools/budget_tool.py)

• **Input Schema:** flight_price, hotel_price_per_night, num_nights, num_days, daily_expense_tier, custom_daily_expense, num_travelers, buffer_percentage.
• **Formula:** Total = (Flights * Travelers) + (Hotel * Nights * Rooms) + (Daily Tier * Days * Travelers) + Buffer.
• **Tiers:** Budget (Rs. 1,000/day), Moderate (Rs. 1,800/day), Luxury (Rs. 3,500/day).

5. Relational Database Design & SQL Practices

To comply with enterprise SQL database standards, all data was normalized into third normal form (3NF) within SQLite (travel_planner.db) with specialized index structures:

Table	Key Columns	Indexes & Optimizations
flights	flight_id, airline, origin, destination, departure_time, arrival_time, duration_minutes, price	idx_flights_route (origin, destination), idx_flights_price, idx_flights_duration
hotels	hotel_id, name, city, stars, price_per_night, amenities	idx_hotels_city, idx_hotels_city_stars (city, stars), idx_hotels_price

places	place_id, name, city, type, rating, description	idx_places_city, idx_places_city_type (city, type), idx_places_rating
saved_itineraries	id, created_at, trip_title, origin, destination, start_date, end_date, days, travelers, total_cost, itinerary_json	idx_saved_dest (destination), primary key index on id

6. Data Ingestion & Dataset Expansion

The project initially received three foundational JSON datasets: flights.json (30 flights), hotels.json (40 hotels), and places.json (40 places). In accordance with the project specification allowing external collection and scraping, an automated enrichment pipeline (scripts/prepare_data.py) was engineered:

- **Original Retention:** 100% of all initial seed records were strictly preserved.
- **Flight Expansion (112 Routes):** Built bidirectional connectivity for all pairs among the 8 hubs: Delhi, Mumbai, Bangalore, Goa, Hyderabad, Chennai, Kolkata, and Jaipur.
- **Hotel Expansion (57 Properties):** Added premier regional lodgings spanning budget hostels (Zostel), business hotels (Bloomrooms), and heritage 5-star palaces (Taj, Oberoi, Leela).
- **Attractions Expansion (96 Iconic POIs):** Added iconic landmarks (e.g., Red Fort, Baga Beach, Dudhsagar Falls, Amber Palace, Victoria Memorial, Charminar).

7. User Interface Development (Streamlit Cloud & CLI)

Streamlit Cloud Application (app.py):

Deployed live on Streamlit Cloud with an intuitive four-tab layout:
1. **Plan My Trip:** Dual-column parameter controls, flight priority toggles, star rating sliders, live multi-tool execution spinner, and expandable agent reasoning trace.
2. **Results Showcase:** Top metric cards, detailed flight/hotel cards, day-by-day sightseeing accordions, live weather forecast cards, and an interactive budget breakdown chart.
3. **Saved Itineraries:** Relational database view enabling users to retrieve, inspect, or delete previously generated trips.
4. **Dataset Explorer:** Live interactive exploration of flights, hotels, and places with city-level filtering.

Command-Line Interface (cli.py):

Provides an interactive console wizard for terminal environments as well as non-interactive execution with command-line arguments (e.g., python cli.py --origin Delhi --destination Goa --days 3 --save).

8. Quality Assurance & Test Verification

A comprehensive test suite was constructed under tests/, achieving a **100% pass rate** across 17 test modules:

Test Module	Test Scope	Status
tests/test_tools.py	Direct and LangChain tool invocations for all 5 tools (Flights, Hotels, Places, Weather, Budget).	PASSED (10/10)
tests/test_database.py	Schema initialization, index utilization, route querying, and itinerary CRUD lifecycle.	PASSED (4/4)
tests/test_agent.py	End-to-end trip planning, Pydantic schema validation, format conformity, and JSON export.	PASSED (3/3)

9. Verification Against Specification Output Rubric

The project specification document mandated a strict format for final itinerary delivery. The agent's generated output below directly verifies full compliance:

```
Your 3-Day Trip to Goa (Nov 13-15)

Flight Selected:
- Air India (Rs. 3,426) - Departs Delhi at 13:00
```

```
Hotel Booked:
- Royal Heritage (Rs. 1,232/night, 5-star)

Weather:
- Day 1: Sunny (31 deg C)
- Day 2: Partly Cloudy (32 deg C)
- Day 3: Light Breeze (30 deg C)

Itinerary:
Day 1: Basilica of Bom Jesus, Fort Aguada
Day 2: Dudhsagar Waterfalls, Baga Beach
Day 3: Calangute Beach Water Sports, Old Goa Heritage Walk

Estimated Total Budget:
- Flight: Rs. 3,426
- Hotel: Rs. 2,464
- Food & Travel: Rs. 2,499
-------------------------------------
Total Cost: Rs. 8,389

Reasoning:
We selected Air India for its budget-friendly fare of Rs. 3,426 and timely departure schedule.
Royal Heritage was chosen as the premier accommodation offering a 5-star experience with top
amenities at Rs. 1,232/night. The daily itinerary aligns outdoor beach and fort visits with
sunny forecast days, reserving indoor museums and markets for pleasant afternoons, keeping the
grand total to Rs. 8,389.
```

10. Conclusion & Final Project Status

The **Agentic AI-Based Travel Planning Assistant Using LangChain** project has been successfully completed, verified, committed to GitHub, and deployed live to production. All primary and secondary objectives - including autonomous multi-tool coordination, free weather API integration, relational database indexing, responsive Streamlit UI, and structured JSON export - have been fully met.

Component	Implementation Detail	Status
Agent Framework	LangChain, LangGraph ReAct + Autonomous Multi-Tool Engine	Completed
Weather Integration	Live Open-Meteo REST API (Zero API key dependency)	Completed
Database & SQL	SQLite 3NF Schema with 6 B-Tree Query Indexes	Completed
Web Interface	Streamlit Cloud Deployment with Reactive State Management	Live
Source Code Repo	GitHub (durg-giri123/Agentic-AI-Travel-Planning-Assistant)	Synced